Fundamentals of Fault Modeling and Structural Testing

Nguyen-Van Thanh-Loc

Email: locnvt@hcmute.edu.vn

Outline

1. Challenges in VLSI Testing

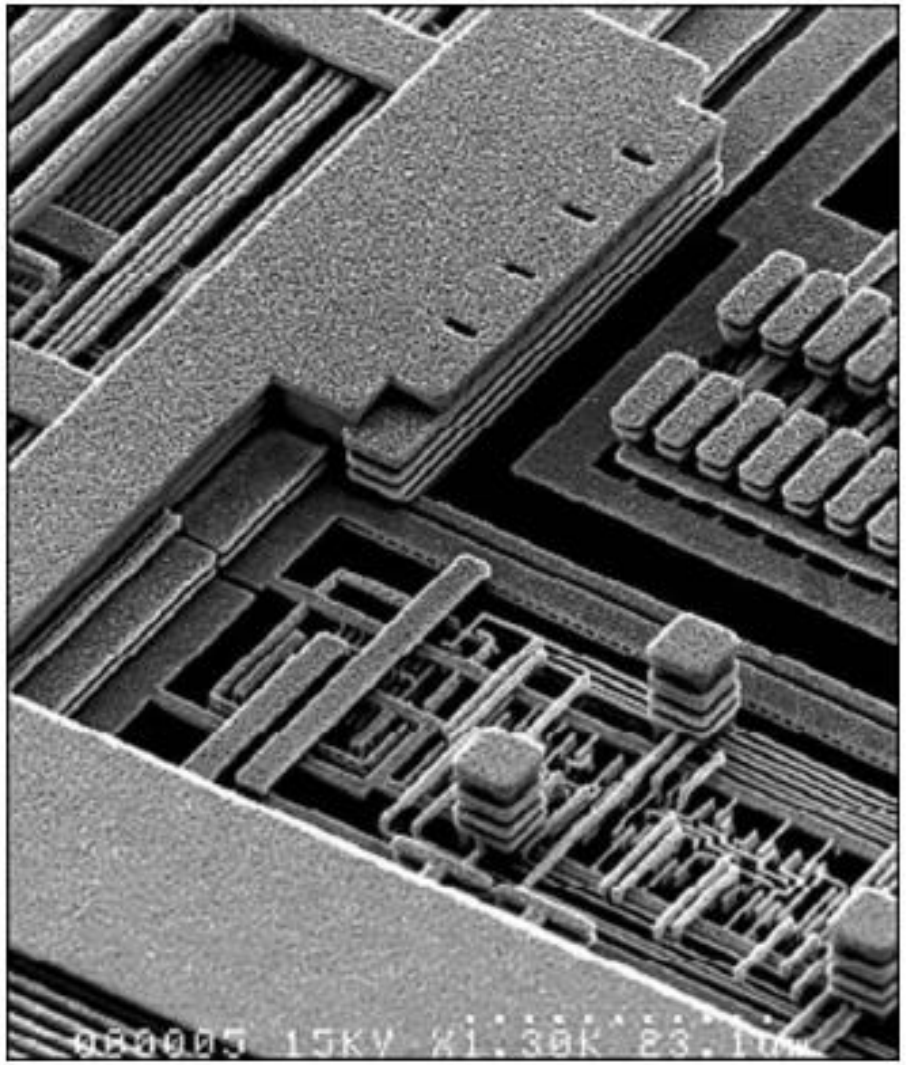
- Fault, Error, and Failure
- Functional and Exhaustive Testing
- Structural Testing and Fault Coverage

2. Fault Models

- Stuck-At Fault Model
- Fault Collapsing
- Transistor Stuck-Open Faults
- Transistor Stuck-Short Faults and IDDQ Testing
- Transistor Fault Collapsing
- Open and Short Faults
- Bridging Fault Model
- Delay Faults and Crosstalk

Challenges in VLSI Testing

- **Any small piece of dust** or abnormality of geometrical shape can **result in a defect.**
- **Defects are caused by process variations** or random localized manufacturing imperfections.



■ **FIGURE 1.6**

IBM CMOS integrated circuit with six levels of interconnections and effective transistor channel length of 0.12 μm [Geppert 1998].

Fault, Error, and Failure

- **A fault** is a representation of **a defect** reflecting a **physical** condition that causes a circuit to **fail** to perform in a required manner.
- A circuit defect may lead to a fault, a fault can cause a circuit error, and a circuit error can result in a system failure.

Functional and Exhaustive Testing

- **To test** a circuit with n inputs and m outputs, a set of **input patterns** is applied to the circuit under test (CUT).
- If the CUT is an **n-input combinational logic circuit**, we can apply all $\mathbf{2^n}$ possible input patterns for testing stuck-at faults; this approach is called exhaustive testing.
- **Unfortunately, exhaustive testing is not practical when n is large.**

Structural Testing and Fault Coverage

- A more practical approach is to select specific test patterns based on **circuit structural information** and **a set of fault models**. This approach is called **structural testing**.
- This measure is called fault coverage and is defined as:

$$\boldsymbol{Fault\ Coverage} = \frac{\boldsymbol{Number\ of\ detected\ faults}}{\boldsymbol{Total\ number\ of\ faults}} \times \mathbf{100} \quad \mathbf{(\%)}$$

- The **goal of test generation** is to find an **efficient set of test vectors** that detects all faults considered for that circuit.

Table 1. Benchmark Results.

		LFSR					Swift				
Benchmark	Gate Count	Coverage (%)	Run-time (sec)	TVCount	Compacted	CPercent (%)	Coverage (%)	Run-time (sec)	TVCount	Compacted	CPercent (%)
c6288	2348	99.99	23.51	100	26	74.00	99.97	23.91	100	25	75.00
c5315	1388	98.09	9.8	100	49	51.00	98.40	9.49	100	52	48.00
c5315a	1332	97.75	9.7	100	46	54.00	98.50	9.71	100	46	54.00
c3540	982	95.32	20.46	300	58	80.67	97.63	20.07	300	65	78.33
s15850a	2792	90.32	119.63	400	91	77.25	90.28	121.83	400	88	78.00
s5378a	1030	92.32	35.04	400	71	82.25	94.74	33.84	400	79	80.25
s1488	610	96.60	12.98	300	57	81.00	95.23	8.51	200	52	74.00
s1423a	481	97.74	8.29	200	37	81.50	95.41	8.42	200	34	83.00
s1423	481	95.54	4.45	100	34	66.00	95.48	9.01	200	31	84.50
APU	5121	83.59	175.86	400	114	71.50	86.017	178.32	400	114	71.50
zigzag	4580	99.99	92.32	100	22	78.00	99.99	95.1	100	22	78.00
RGB2YCBCR	3568	91.40	117.33	400	62	84.50	91.89	118.97	400	66	83.50
cpu6502	2395	92.70	71.99	400	113	71.75	91.83	71.58	400	115	71.25
can	2248	91.73	73.74	400	120	70.00	90.75	75.15	400	112	72.00
cic_interpolator	1026	98.18	31.89	400	23	94.25	98.53	23.46	300	23	92.33

[1] Gaber, M., Abdelatty, M., & Shalan, M. (2019, November). Fault, an open source DFT toolchain. In *Proceedings of the Workshop on Open-Source EDA Technology (WOSET19)*.

Fault Models

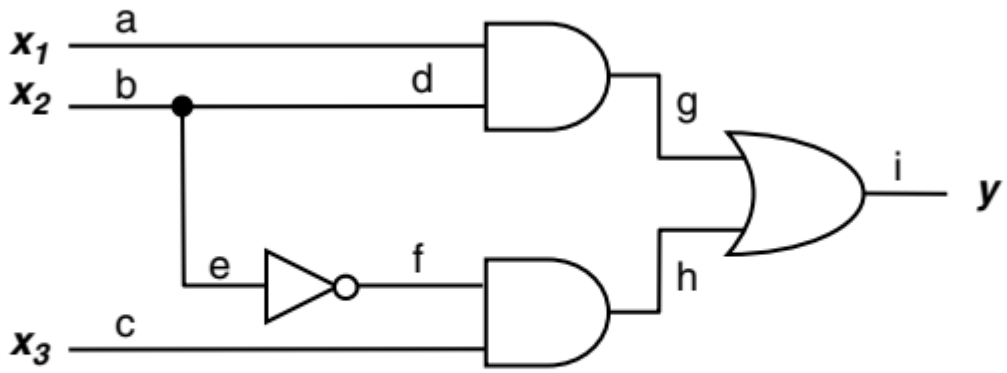
- **Fault models** are necessary for **generating and evaluating** a set of test vectors.
- Generally, a good fault model should satisfy **two** criteria:
  - (1) It should accurately **reflect** the behavior of defects.
  - (2) it should be computationally **efficient** in terms of fault **simulation** and test pattern **generation**.
- Fortunately, it has been shown that high fault coverage obtained under the single-fault assumption will result in high fault coverage for the multiple-fault model; therefore, the **single-fault** assumption **is typically used** for test generation and evaluation.

Physical Defect / Failure Mechanism	Fault Model Used for Testing
Open circuit / broken interconnect	Open Fault Model
Signal line shorted to GND	Stuck-at-0 (SA0) Fault Model
Signal line shorted to VDD	Stuck-at-1 (SA1) Fault Model
Short circuit between two signal lines	Bridging Fault Model
Transistor stuck in the OFF state	Transistor Stuck-Open Fault Model
Transistor stuck in the ON state	Transistor Stuck-Short Fault Model
Excessive delay at a circuit node	Transition Fault Model
Excessive cumulative delay along a signal path	Path-Delay Fault Model
Interconnect coupling / crosstalk causing timing degradation	Delay Fault / Crosstalk Fault Model

Stuck-At Fault Model (1/3)

- A stuck-at fault transforms the correct value on the faulty signal line to appear to be stuck at **a constant logic value**, either **a logic 0** or **a logic 1**, referred to as **stuck-at-0 (SA0)** or **stuck-at-1 (SA1),** respectively.
- Consider the example circuit shown in Figure 1.7, where the nine signal lines representing potential fault sites are labeled alphabetically. There are 18 **(2 × 9)** possible faulty circuits under the single-fault assumption.
- This can be seen in Table 1.1, where SA0 on fanout branch line d behaves differently from SA0 on fanout branch line e, while the single SA0 fault on the fanout source line b behaves as if both fanout branches line d and line e are SA0.

Stuck-At Fault Model (2/3)



■ **FIGURE 1.7**

Example circuit.

TABLE 1.1 ■ Truth Tables for Fault-Free and Faulty Circuits of Figure 1.7

$x_1x_2x_3$	000	001	010	011	100	101	110	111
y	0	1	0	0	0	1	1	**1**
a SA0	0	1	0	0	0	1	0	**0**
a SA1	0	1	1	1	0	1	1	**1**
b SA0	0	1	0	1	0	1	0	**1**
b SA1	0	0	0	0	1	1	1	**1**
c SA0	0	0	0	0	0	0	1	**1**
c SA1	1	1	0	0	1	1	1	**1**
d SA0	0	1	0	0	0	1	0	**0**
d SA1	0	1	0	0	1	1	1	**1**
e SA0	0	1	0	1	0	1	1	**1**
e SA1	0	0	0	0	0	0	1	**1**
f SA0	0	0	0	0	0	0	1	**1**
f SA1	0	1	0	1	0	1	1	**1**
g SA0	0	1	0	0	0	1	0	**0**
g SA1	1	1	1	1	1	1	1	**1**
h SA0	0	0	0	0	0	0	1	**1**
h SA1	1	1	1	1	1	1	1	**1**
i SA0	0	0	0	0	0	0	0	**0**
i SA1	**1**	**1**	**1**	**1**	**1**	**1**	**1**	**1**

Stuck-At Fault Model (3/3)

- The input values for the highlighted truth table entries represent valid test vectors to detect the associated stuck-at faults.
- Test vectors 011 and 100 must be included in any set of test vectors that will obtain 100% fault coverage for this circuit.
- This set of **four test vectors** obtains 100% single stuck-at fault coverage for this circuit.

Fault Collapsing

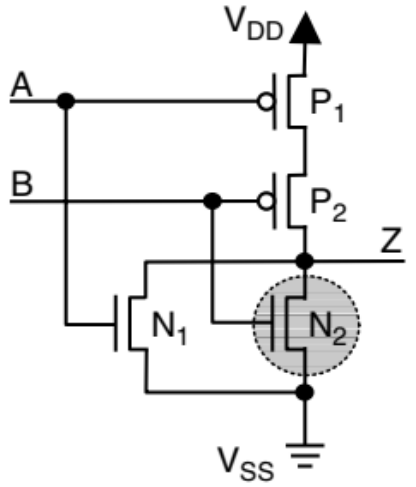
- One fault from each set can be used to represent all of the equivalent faults in that set.
- Stuck-at fault collapsing typically reduces the total number of faults by 50 to 60%.
- The number of collapsed stuck-at faults in any combinational circuit is given by:

$$Number\ of\ collapsed\ faults = 2(POs + Fanout\ Stems) + Gate\ Inputs - Inverters$$

- The total number of gate inputs is 7, including the input to the one inverter; therefore, the number of collapsed faults = 2 × (1 + 1) + 7 − 1 = 10.

Transistor Stuck-Open Faults

- The stuck-at fault model cannot accurately reflect the behavior of stuck-open and stuck-short faults in CMOS logic circuits because of the multiple transistors used to construct CMOS logic gates.
- In order to detect this fault, an ordered sequence of two test vectors AB = 00 → 01 is required.



Transistor Stuck-Short Faults and IDDQ Testing

- Stuck-short faults will produce a conducting path between VDD and VSS.
- This creates a voltage divider at the output node Z where the logic level voltage will be a function of the resistances of the conducting transistors.
- Stuck-short transistor faults may be detected by monitoring the power supply current during steady state, referred to as IDDQ.

Transistor Fault Collapsing

- Stuck-open faults in a group of series transistors are indistinguishable, while stuck-short faults in a group of parallel transistors are also indistinguishable; therefore, fault collapsing can be applied to transistor-level circuits.
- **The number of collapsed transistor faults in a circuit is given by:**

$$N_{collapsed} = 2T - T_S + G_S - T_P + G_P$$

- Because both N1 and N2 stuck-short faults as well as P1 and P2 stuck-open faults can be tested by the same test set, the collapsed fault count is 6.

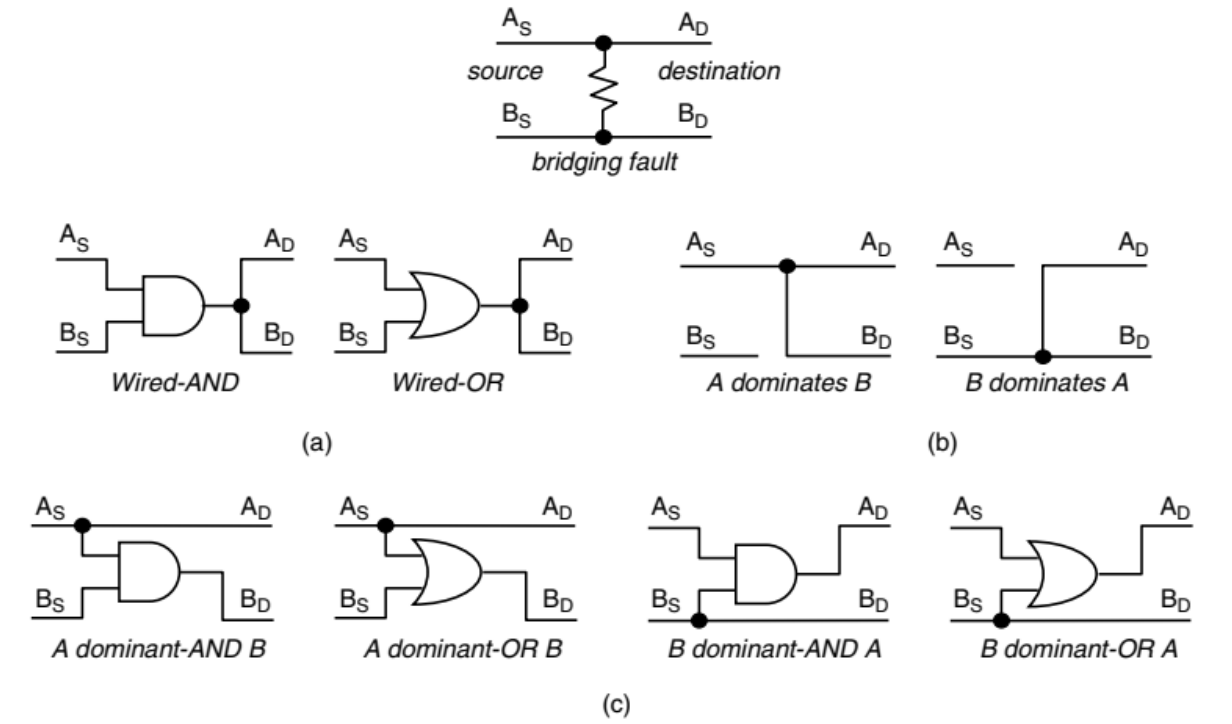
Open and Short Faults

- Defects in VLSI devices can include opens and shorts in the wires that interconnect the transistors forming the circuit.
- Opens in wires tend to behave like transistor stuck-open faults when the faulty wire segment is interconnecting transistors to form gates, and like stuck-at faults when interconnecting gates.
- Therefore, a set of test vectors that provide high stuck-at fault coverage and high transistor fault coverage will also detect open faults.

Bridging Fault Model (1/4)

- A short between two elements is commonly referred to as a bridging fault.
- When two signal wires are shorted together, bridging fault models are required.
- The wired-AND bridging fault means the signal net formed by the two shorted lines will take on a logic 0 if either shorted line is sourcing a logic 0, while the wired-OR bridging fault means the signal net will take on a logic 1 if either of the two lines is sourcing a logic 1.

Bridging Fault Model (2/4)



■ **FIGURE 1.9**

Bridging fault models.

Bridging Fault Model (3/4)

TABLE 1.3 ■ Truth Tables for Bridging Fault Models of Figure 1.9

$A_S B_S$	0	0	0	1	1	0	**1**	**1**
$A_D B_D$	0	0	0	1	1	0	**1**	**1**
Wired-AND	0	0	0	0	0	0	**1**	**1**
Wired-OR	0	0	1	1	1	1	**1**	**1**
A dominates B	0	0	0	0	1	1	**1**	**1**
B dominates A	0	0	1	1	0	0	**1**	**1**
A dominant-AND B	0	0	0	0	1	0	**1**	**1**
B dominant-AND A	0	0	0	1	0	0	**1**	**1**
A dominant-OR B	0	0	0	1	1	1	**1**	**1**
B dominant-OR A	**0**	**0**	**1**	**1**	**1**	**0**	**1**	**1**

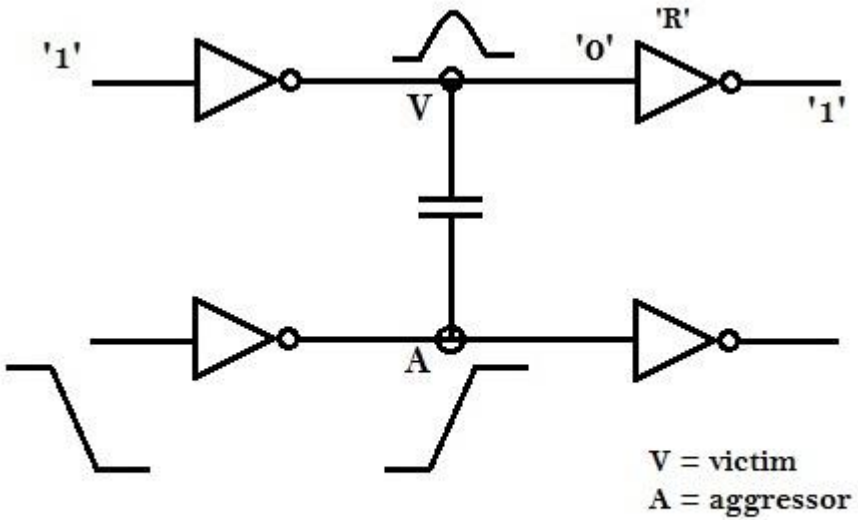
Bridging Fault Model (4/4)

- The wired-AND/wired-OR bridging fault model was originally developed for bipolar VLSI and does not accurately reflect the behavior of bridging faults typically found in CMOS devices; therefore, the dominant bridging fault model was proposed for CMOS VLSI.
- It has been shown that a set of test vectors that detects all dominant bridging faults is also guaranteed to detect all wired-AND and wired-OR bridging faults.
- One solution to this problem is to extract likely bridging fault sites from the physical design after physical layout.

Delay Faults and Crosstalk (1/3)

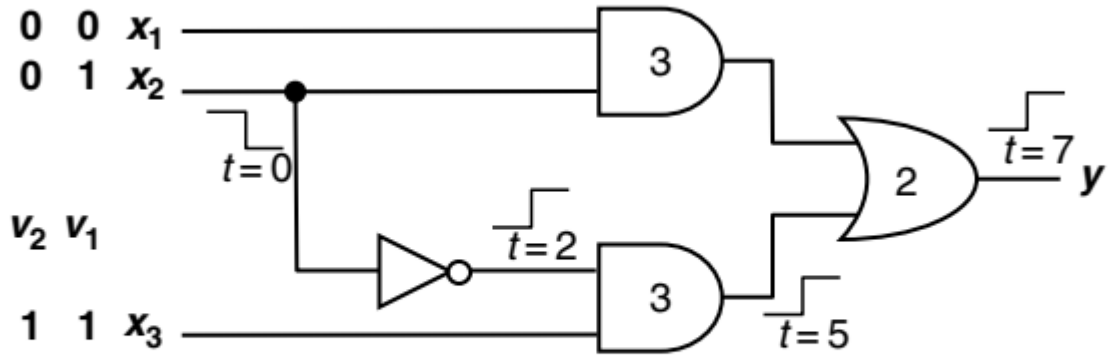
- A **delay fault** causes excessive delay along a path such that the total **propagation delay falls** outside the specified limit.
- **The path-delay fault model** is more practical for testing than the gate-delay fault (or the transition fault) model because it considers the cumulative propagation delay along a signal path through the CUT.
- The use of **nanometer technologies** increases **cross-coupling capacitance** and inductance between interconnects, leading to severe **crosstalk effects** that may result in improper functioning of a chip.

Delay Faults and Crosstalk (2/3)



Crosstalk noise leading to logic failures

Delay Faults and Crosstalk (3/3)

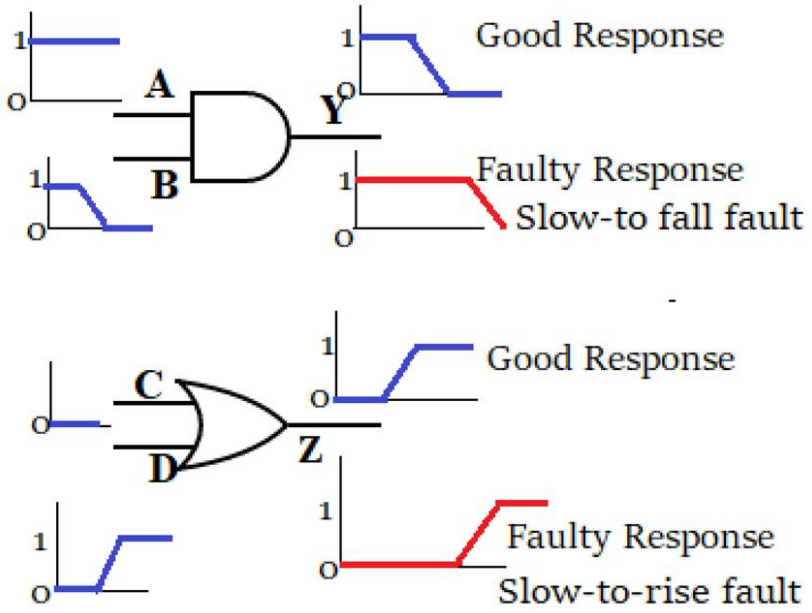


■ **FIGURE 1.10**

Path-delay fault test.

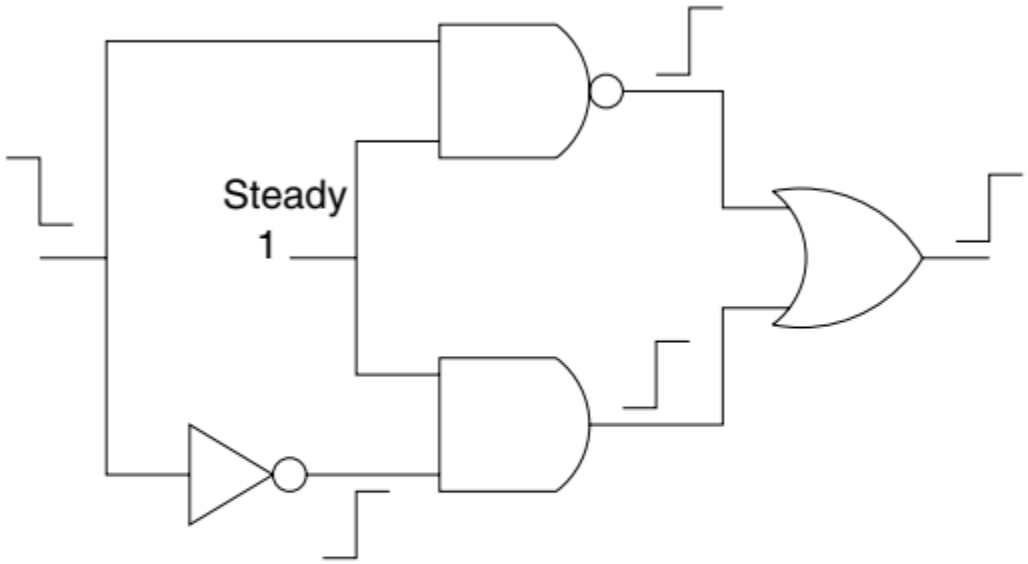
Transition Faults

- Transition fault models a delay defect at a circuit node.
- Two types: slow-to-rise and slow-to-fall.
- The fault causes a 0→1 or 1→0 transition to occur too slowly.
- A transition test requires two vectors $V1, V2$: initialization and launch.
- The fault effect must propagate to an observable point (PO/FF)



Ref: 4.10 page 232

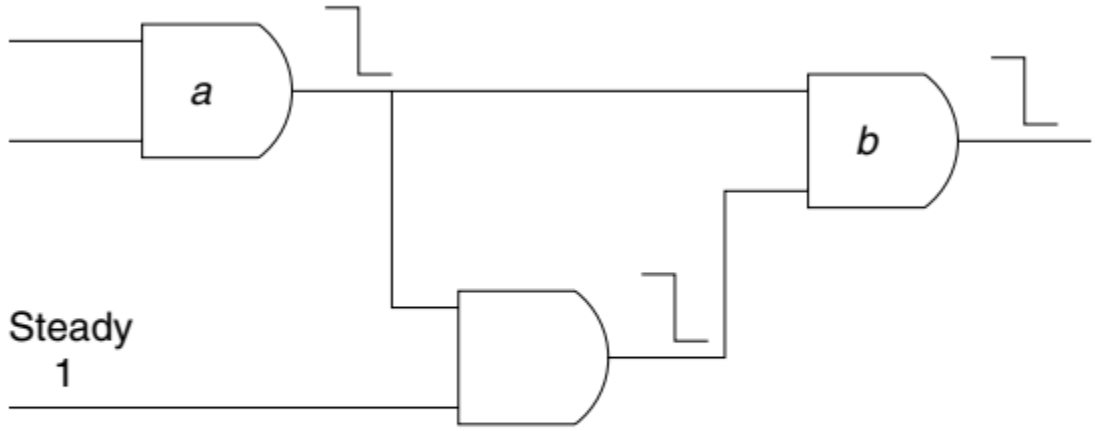
Transition Faults



■ **FIGURE 4.52**

Slow-to-rise transition at the input of an OR gate.

Transition Faults

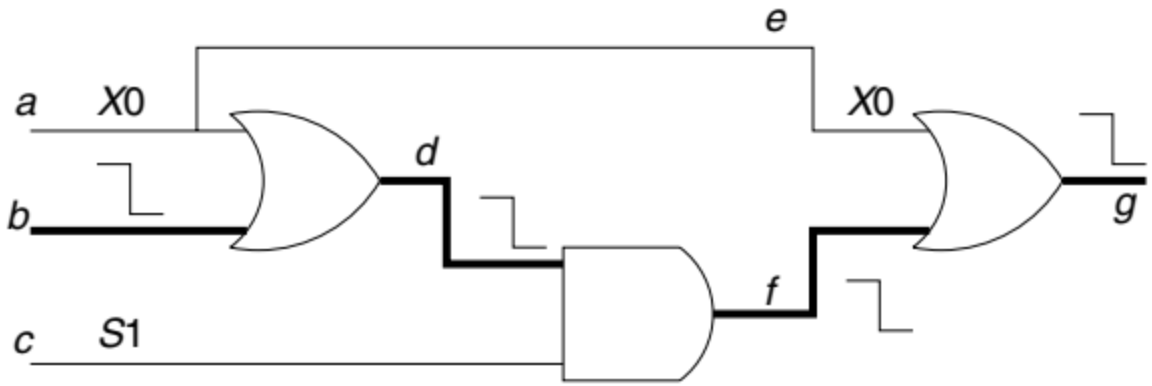


■ **FIGURE 4.53**

Slow-to-fall transition propagation example.

Path-delay faults

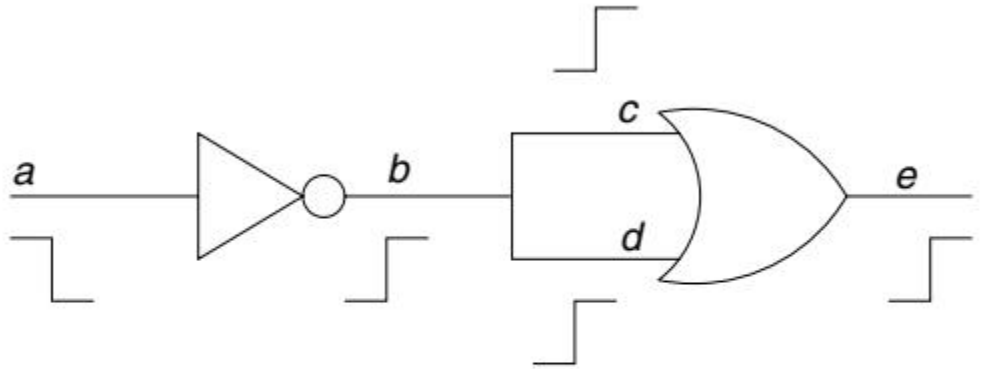
- The **path-delay fault** model considers the **cumulative** effect of the delays along a **specific path.**



■ **FIGURE 4.49**

A robustly testable path.

Path-delay faults

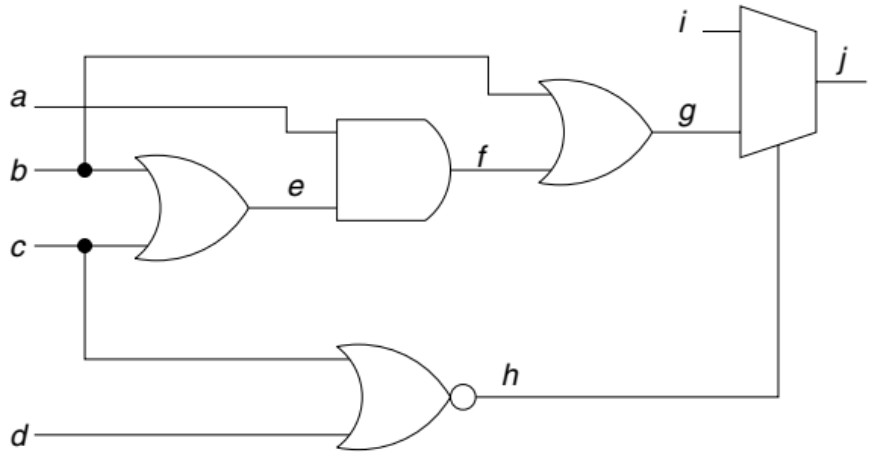


■ **FIGURE 4.48**

A statically unsensitizable but not false path.

Path-delay faults

4.31 (Path-Delay ATPG) Consider the circuit fragment shown in Figure 4.62.



■ **FIGURE 4.62**

Example circuit.

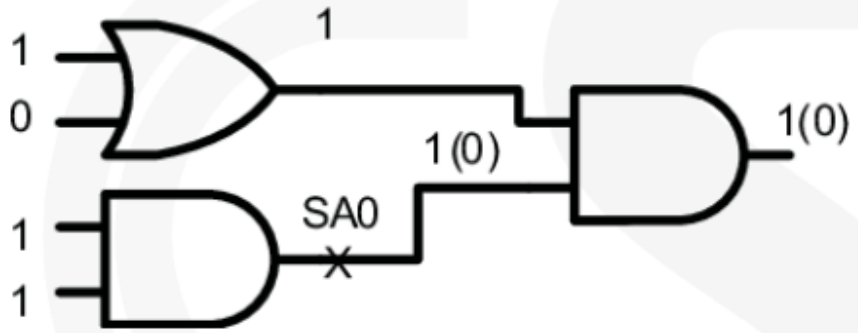
a. Generate all paths in this circuit. How many paths are there in this circuit?
b. Which paths are functionally unsensitizable?
c. For those sensitizable paths, which ones are robustly testable, and which ones are nonrobustly testable?

DC test and AC test

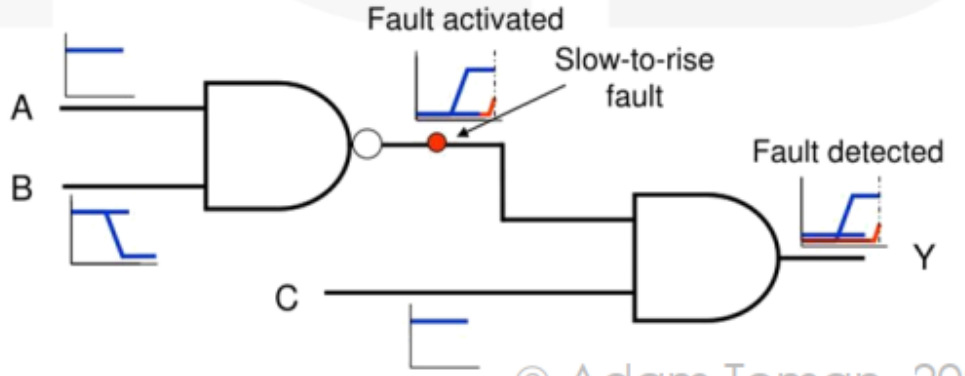
- This is particularly important for **slow-speed testing (DC test)** of structural faults, such as **stuck-at faults** and bridging faults, as well as **at-speed testing (AC test)** of timing-related delay faults, such as **path-delay faults** and **transition faults**.

Commonly used Fault Models

- **Stuck At Fault**
  - Assume all failures cause nodes to be “stuck-at” 0 or 1
  - Static model (as opposed to at-speed test)
  - Independent of process technology
  - Not quite true, but works well in practice



- **Transition/Delay Fault**
  - “Slow to Rise” or “Slow to Fall” fault
  - Signal propagation delays that are outside the circuit specifications
  - Dynamic model, tested at-speed

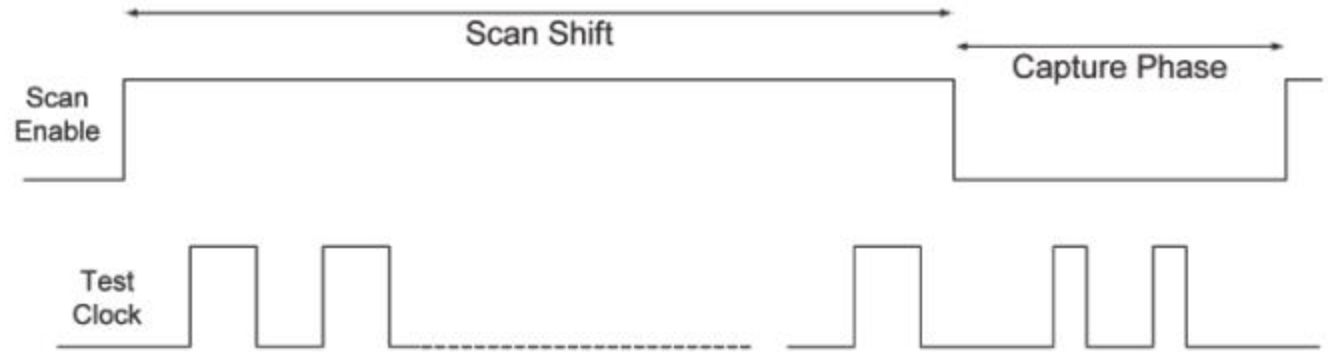


- **Other fault models**
  - Transistor opens and shorts
  - Bridging faults

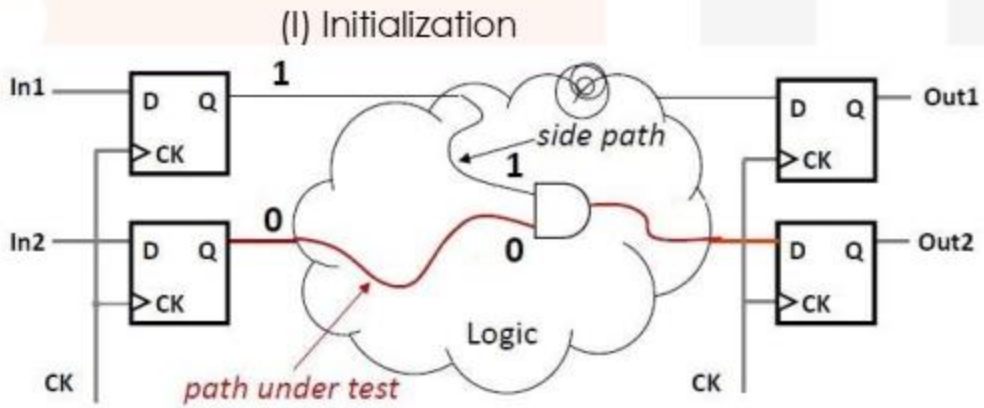
© Adam Teman, 2020

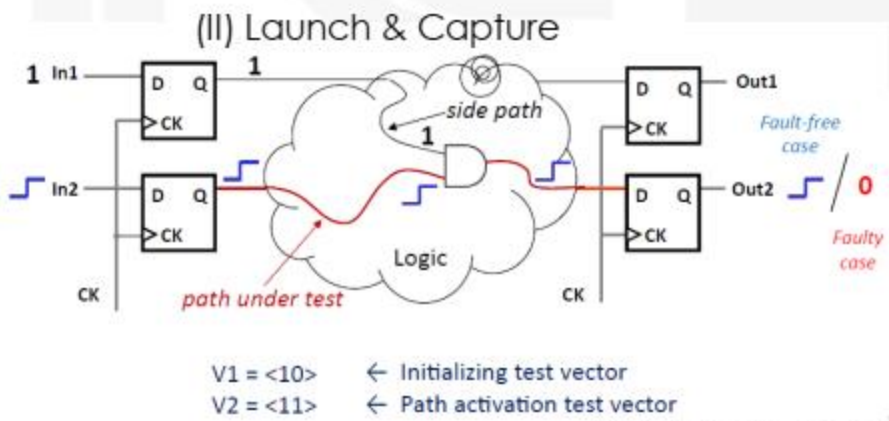
AC Scan (at-speed test)

- **Used to verify:**
  - Transition Faults
  - Delay Faults
- **A path delay fault requires a pair of subsequent vectors to be detected**
  - The first test vector initializes the circuit
  - The second test vector activates the path under test



Source: Atrenta





© Adam Teman, 2020